ANALISIS DAN PEMBUKTIAN REPRESENTASI CITRA DIGITAL MENGGUNAKAN PYTHON

Nama : Ade Ayu Wandira

NIM : 24260007

Semester : 5

Mata Kuliah : Aplikasi Berbasis Service

A. Pendahuluan

Foto digital yang tampak sebagai gambar utuh pada layar sebenarnya tersusun dari titik-titik kecil yang disebut pixel. Setiap pixel menyimpan nilai angka yang menentukan warnanya. Pada citra berwarna dengan mode RGB, satu pixel direpresentasikan oleh tiga angka, yaitu komponen Red (R), Green (G), dan Blue (B).

Analisis ini dilakukan menggunakan Python dengan tujuan membuktikan bahwa foto digital tersusun dari pixel yang memiliki nilai numerik dan dapat direpresentasikan sebagai array angka. Pembuktian dilakukan secara bertahap: membaca dan menampilkan foto, memeriksa informasi dan metadata EXIF, membaca metadata GPS dan mengubah koordinatnya menjadi derajat desimal, mengubah foto menjadi array, melihat nilai pixel secara langsung, menentukan lima koordinat pixel untuk dibaca nilai RGB dan warnanya, menandai letak kelima pixel tersebut pada foto asli, menampilkan sebagian pixel sebagai matriks angka, membentuk kembali gambar dari array tersebut, kemudian memeriksa apakah gambar hasil rekonstruksi identik dengan gambar aslinya.

Seluruh kode, hasil, dan angka pada laporan ini diambil dari notebook `analisis_citra.ipynb`, yang terdiri dari 14 tahap (codingan nomor 1 sampai nomor 14). Tampilan kode pada laporan diberi nomor baris agar mudah dicocokkan dengan penjelasan, dan tampilan kode serta hasil program menggunakan tema gelap seperti pada notebook.

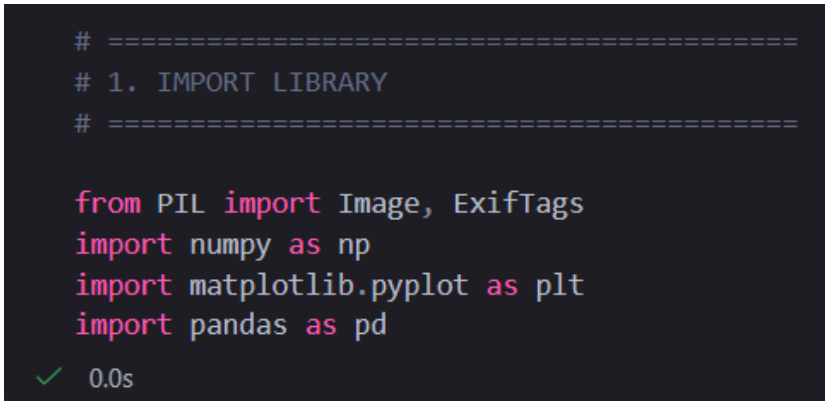
B. Implementasi dan Pembahasan

Codingan Nomor 1: Import Library

1. Tujuan codingan

Menyiapkan seluruh library yang dibutuhkan pada tahap analisis berikutnya, yaitu untuk membaca gambar, membaca metadata, mengolah array angka, menampilkan gambar, dan menyusun tabel.

2. Screenshot kode program



Gambar 1. Kode program nomor 1 (import library)

3. Penjelasan setiap baris kode

Baris 1–3

```
# ====...
# 1. IMPORT LIBRARY
# ====...
```

Baris yang diawali tanda `#` adalah komentar. Python tidak mengeksekusinya; fungsinya hanya sebagai judul penanda bahwa blok kode ini adalah tahap nomor 1.

Baris 5

```
from PIL import Image, ExifTags
```

Mengimpor dua komponen dari library Pillow (PIL). `Image` dipakai untuk membuka, membaca, dan membentuk objek gambar, sedangkan `ExifTags` berisi daftar nama tag metadata EXIF dan GPS sehingga ID metadata yang berupa angka dapat diterjemahkan menjadi nama yang mudah dibaca.

Baris 6

```
import numpy as np
```

Mengimpor NumPy dengan alias `np`. NumPy digunakan untuk mengolah data numerik berbentuk array, termasuk array yang menyimpan nilai pixel gambar.

Baris 7

`import matplotlib.pyplot as plt`

Mengimpor modul `pyplot` dari Matplotlib dengan alias `plt`. Modul ini dipakai untuk menampilkan gambar dan membuat visualisasi.

Baris 8

`import pandas as pd`

Mengimpor Pandas dengan alias `pd`. Pandas digunakan untuk menyusun dan menampilkan data dalam bentuk tabel (DataFrame), misalnya tabel metadata, tabel koordinat GPS, dan tabel nilai RGB.

4. Hasil program

Tahap ini tidak menghasilkan output yang terlihat. Pengaruhnya adalah seluruh library tersedia untuk dipakai pada nomor 2 sampai nomor 14.

5. Analisis hasil

Pembagian tugas library sudah jelas: Pillow untuk akses file gambar dan metadata, NumPy untuk representasi angka, Matplotlib untuk tampilan visual, dan Pandas untuk tabel. Tanpa tahap ini, fungsi seperti `Image.open()`, `np.array()`, `plt.imshow()`, dan `pd.DataFrame()` pada tahap berikutnya tidak dapat dipanggil.

Codingan Nomor 2: Membaca Citra

1. Tujuan codingan

Membuka file foto dan memuatnya ke dalam Python agar dapat dianalisis pada tahap selanjutnya.

2. Screenshot kode program

```
# ============================================
# 2. MEMBACA CITRA
# ============================================

gambar = Image.open("foto/IMG_20240413_053916.jpg")
✓ 0.0s
```

Gambar 2. Kode program nomor 2 (membaca citra)

3. Penjelasan setiap baris kode

Baris 1–3

```
# ====...
# 2. MEMBACA CITRA
# ====...
```

Komentar penanda tahap nomor 2.

Baris 5

```
gambar = Image.open("foto/IMG_20240413_053916.jpg")
```

`Image.open()` adalah fungsi Pillow untuk membuka file gambar. Argumennya adalah lokasi file, yaitu `"foto/IMG_20240413_053916.jpg"`: file berada di dalam folder `foto` relatif terhadap lokasi notebook. Nama file mengikuti pola penamaan kamera ponsel (`IMG_` diikuti tanggal 20240413 dan jam 053916), yang sejalan dengan data `DateTime` pada metadata EXIF di nomor 4. Hasil pembukaan disimpan pada variabel `gambar`, yang berupa objek gambar Pillow dan akan dipakai pada hampir seluruh tahap berikutnya.

4. Hasil program

Tahap ini tidak menampilkan output. Hasilnya adalah variabel `gambar` yang sudah siap dipakai untuk ditampilkan (nomor 3), dibaca informasinya (nomor 4), dibaca metadata GPS-nya (nomor 5 dan 6), dan diubah menjadi array (nomor 7).

5. Analisis hasil

Pada tahap ini foto belum berupa angka yang dapat dihitung, melainkan baru berupa objek gambar yang dikenali Pillow. Karena tidak muncul pesan kesalahan, dapat disimpulkan file ditemukan di lokasi yang ditulis dan formatnya dapat dibaca. Pengubahan objek ini menjadi array angka dilakukan pada nomor 7.

Codingan Nomor 3: Menampilkan Citra

1. Tujuan codingan

Menampilkan foto yang telah dibaca menggunakan Matplotlib agar objek yang dianalisis dapat dilihat secara visual.

2. Screenshot kode program

```
# ========================================
# 3. MENAMPILKAN CITRA
# ========================================

plt.imshow(gambar)
plt.title("Citra Digital yang Digunakan dalam Analisis")
plt.axis("off");
```

✓ 0.7s

Gambar 3. Kode program nomor 3 (menampilkan citra)

3. Penjelasan setiap baris kode

Baris 1–3

```
# ====...
# 3. MENAMPILKAN CITRA
# ====...
```

Komentar penanda tahap nomor 3.

Baris 5

```
plt.imshow(gambar)
```

`plt.imshow()` menampilkan gambar pada area plot Matplotlib. Argumen `gambar` adalah objek yang dibuka pada nomor 2.

Baris 6

```
plt.title("Citra Digital yang Digunakan dalam Analisis")
```

Memberi judul di atas gambar sehingga tampilan memiliki keterangan.

Baris 7

```
plt.axis("off");
```

Menyembunyikan sumbu koordinat (angka dan garis di tepi plot) agar gambar terlihat lebih bersih. Tanda titik koma (`;`) di akhir baris mencegah notebook mencetak teks tambahan dari nilai yang dikembalikan fungsi, sehingga yang tampil hanya gambarnya.

4. Hasil program



Gambar 4. Hasil tampilan citra pada nomor 3

5. Analisis hasil

Foto tampil dengan judul yang diberikan: bagian dalam dapur berdinding anyaman bambu dengan tungku, peralatan masak, dan sebuah drum biru. Tampilan ini adalah wujud visual dari data yang akan diuraikan menjadi angka pada tahap berikutnya. Sampai tahap ini kita baru melihat gambar; bukti bahwa gambar tersusun dari pixel bernilai angka mulai ditunjukkan pada nomor 4 dan dibuktikan melalui array pada nomor 7.

Codingan Nomor 4: Informasi Gambar

1. Tujuan codingan

Mengetahui informasi dasar gambar (format, ukuran, mode warna, dan jumlah kanal) serta membaca metadata EXIF yang tersimpan pada file foto.

2. Screenshot kode program

```
# ========================================
# 4. INFORMASI GAMBAR
# ========================================

# Membaca metadata EXIF
exif_data = gambar.getexif()

# Mengecek jumlah metadata yang ditemukan
print("Jumlah data EXIF:", len(exif_data))

# Menampilkan metadata dalam bentuk tabel
data_tabel = []

for tag_id, value in exif_data.items():
    tag_name = ExifTags.TAGS.get(tag_id, tag_id)
    data_tabel.append([tag_name, value])

# Informasi dasar gambar
print("\nInformasi Dasar Gambar")
print("Format gambar :", gambar.format)
print("Ukuran gambar :", gambar.size)
print("Mode warna    :", gambar.mode)
print("Jumlah kanal  :", len(gambar.getbands()))

pd.DataFrame(data_tabel, columns=["Metadata", "Nilai"])
✓ 0.0s
```

Gambar 5. Kode program nomor 4 (informasi gambar dan metadata EXIF)

3. Penjelasan setiap baris kode

Baris 1–3

```
# ====...
# 4. INFORMASI GAMBAR
# ====...
```

Komentar penanda tahap nomor 4.

Baris 5–6

```
# Membaca metadata EXIF
exif_data = gambar.getexif()
```

`getexif()` adalah metode Pillow untuk mengambil metadata EXIF, yaitu data tambahan yang disisipkan kamera ke dalam file foto. Hasilnya disimpan pada `exif_data` dengan struktur mirip kamus (dictionary): kunci berupa ID tag angka dan nilainya berupa isi metadata.

Baris 8–9

```
# Mengecek jumlah metadata yang ditemukan
print("Jumlah data EXIF:", len(exif_data))
```

`len(exif_data)` menghitung banyaknya entri metadata, kemudian `print()` menampilkannya. Hasilnya 14 data.

Baris 11–12

```
# Menampilkan metadata dalam bentuk tabel
data_tabel = []
```

Membuat daftar kosong bernama `data_tabel` sebagai wadah pasangan nama metadata dan nilainya sebelum diubah menjadi tabel.

Baris 14

```
for tag_id, value in exif_data.items():
```

Perulangan yang membaca seluruh isi `exif_data`. Metode `items()` memberikan pasangan (ID tag, nilai) dan setiap pasangan ditampung pada variabel `tag_id` dan `value` secara bergantian.

Baris 15

```
    tag_name = ExifTags.TAGS.get(tag_id, tag_id)
```

`ExifTags.TAGS` adalah kamus yang memetakan ID angka ke nama tag, misalnya ID untuk merek kamera menjadi `Make`. Metode `get(tag_id, tag_id)` mengambil nama tag; jika ID tidak ditemukan, yang dipakai adalah ID itu sendiri sehingga program tidak berhenti dengan kesalahan.

Baris 16

```
    data_tabel.append([tag_name, value])
```

Menambahkan satu baris berisi nama metadata dan nilainya ke daftar `data_tabel`. Setelah perulangan selesai, daftar ini memuat seluruh metadata.

Baris 18–19

```
# Informasi dasar gambar
print("\nInformasi Dasar Gambar")
```

Mencetak judul bagian. Karakter `\n` menambah baris kosong sebelum judul agar keluaran lebih rapi.

Baris 20

```
print("Format gambar :", gambar.format)
```

`gambar.format` memberi format file asli, yaitu JPEG.

Baris 21

```
print("Ukuran gambar :", gambar.size)
```

`gambar.size` memberi ukuran dalam urutan (lebar, tinggi), yaitu (3264, 2448) pixel.

Baris 22

```
print("Mode warna    :", gambar.mode)
```

`gambar.mode` memberi mode warna, yaitu RGB.

Baris 23

```
print("Jumlah kanal  :", len(gambar.getbands()))
```

`getbands()` mengembalikan nama kanal warna (R, G, B), dan `len()` menghitung jumlahnya, yaitu 3 kanal.

Baris 25

```
pd.DataFrame(data_tabel, columns=["Metadata", "Nilai"])
```

Mengubah `data_tabel` menjadi tabel Pandas dengan dua kolom bernama `Metadata` dan `Nilai`. Karena berada di baris terakhir sel, tabel ditampilkan otomatis oleh notebook tanpa perintah `print()`.

4. Hasil program

```
Jumlah data EXIF: 14

Informasi Dasar Gambar
Format gambar : JPEG
Ukuran gambar : (3264, 2448)
Mode warna    : RGB
Jumlah kanal  : 3
```

Gambar 6. Hasil informasi dasar gambar pada nomor 4

	Metadata	Nilai
0	ImageWidth	3264
1	ImageLength	2448
2	GPSInfo	1103
3	ResolutionUnit	2
4	ExifOffset	346
5	ImageDescription	
6	Make	vivo
7	Model	V2252
8	Software	MediaTek Camera Application
9	Orientation	1
10	DateTime	2024:04:13 05:39:17
11	YCbCrPositioning	2
12	XResolution	72.0
13	YResolution	72.0

Gambar 7. Tabel metadata EXIF pada nomor 4

5. Analisis hasil

Foto berformat JPEG, berukuran 3264 × 2448 pixel (lebar × tinggi), bermode RGB, dan memiliki 3 kanal warna. Artinya, setiap pixel nantinya akan memiliki tiga nilai angka.

Terdapat 14 metadata EXIF. Beberapa yang paling informatif adalah `Make` bernilai vivo dan `Model` bernilai V2252 (perangkat pengambil foto), `Software` bernilai MediaTek Camera Application, serta `DateTime` bernilai 2024:04:13 05:39:17 (waktu pengambilan foto). `ImageWidth` 3264 dan `ImageLength` 2448 sama dengan ukuran yang dibaca dari `gambar.size`, sedangkan `XResolution` dan `YResolution` bernilai 72.0. Beberapa tag

lain, seperti `GPSInfo` dan `ExifOffset`, tampil sebagai angka karena nilainya berupa penunjuk ke bagian data lain dalam file, dan `ImageDescription` kosong. Nilai 1103 pada `GPSInfo` menandakan bahwa foto ini memiliki blok data GPS, yang isinya dibaca pada nomor 5. Informasi ini juga menjadi dasar sebelum gambar diubah menjadi array pada nomor 7.

Codingan Nomor 5: Membaca Metadata GPS

1. Tujuan codingan

Membaca metadata GPS yang tersimpan pada EXIF foto dan menampilkan setiap tag GPS beserta nilainya. Pada nomor 4, tag `GPSInfo` hanya tampil sebagai angka penunjuk, sehingga isinya baru dapat dilihat pada tahap ini.

2. Screenshot kode program

```
# ========================================
# 5. MEMBACA METADATA GPS
# ========================================

# Mengambil data metadata EXIF dari gambar
exif_data = gambar.getexif()

# Mengambil informasi GPS dari metadata EXIF
gps_data = exif_data.get_ifd(ExifTags.IFD.GPSInfo)

# Memeriksa ketersediaan metadata GPS
if gps_data:

    # Menampilkan metadata GPS yang ditemukan
    for tag_id, value in gps_data.items():
        tag_name = ExifTags.GPSTAGS.get(tag_id, tag_id)
        print(f"{tag_name}: {value}")

else:
    print("Data koordinat GPS tidak ditemukan.")
✓ 0.0s
```

Gambar 8. Kode program nomor 5 (membaca metadata GPS)

3. Penjelasan setiap baris kode

Baris 1–3

```
# ====...
# 5. MEMBACA METADATA GPS
# ====...
```

Komentar penanda tahap nomor 5.

Baris 5–6

```
# Mengambil data metadata EXIF dari gambar
exif_data = gambar.getexif()
```

Membaca metadata EXIF dari `gambar` dan menyimpannya pada `exif_data`. Perintah ini sama dengan yang dipakai pada nomor 4, dituliskan kembali di sel ini sehingga tahap pembacaan GPS memiliki langkah pengambilan datanya sendiri.

Baris 8–9

```
# Mengambil informasi GPS dari metadata EXIF
gps_data = exif_data.get_ifd(ExifTags.IFD.GPSInfo)
```

`get_ifd()` mengambil satu bagian khusus (IFD) dari metadata EXIF. `ExifTags.IFD.GPSInfo` menunjuk bagian GPS, yaitu bagian yang pada nomor 4 hanya tampil sebagai angka 1103. Hasilnya berupa kamus berisi ID tag GPS dan nilainya, disimpan pada `gps_data`. Jika foto tidak memiliki data GPS, kamus ini kosong.

Baris 11–12

```
# Memeriksa ketersediaan metadata GPS
if gps_data:
```

Memeriksa apakah data GPS tersedia. Kamus yang berisi dianggap benar (`True`) sehingga blok di bawahnya dijalankan, sedangkan kamus kosong dianggap salah (`False`) sehingga blok `else` yang dijalankan.

Baris 14–15

```
    # Menampilkan metadata GPS yang ditemukan
    for tag_id, value in gps_data.items():
```

Perulangan yang membaca setiap pasangan (ID tag, nilai) dari `gps_data`, dengan cara yang sama seperti pada nomor 4.

Baris 16

```
        tag_name = ExifTags.GPSTAGS.get(tag_id, tag_id)
```

`ExifTags.GPSTAGS` adalah kamus yang memetakan ID tag GPS ke namanya, misalnya ID 2 menjadi `GPSLatitude`. Jika ID tidak ditemukan, yang dipakai adalah ID itu sendiri.

Baris 17

```
        print(f"{tag_name}: {value}")
```

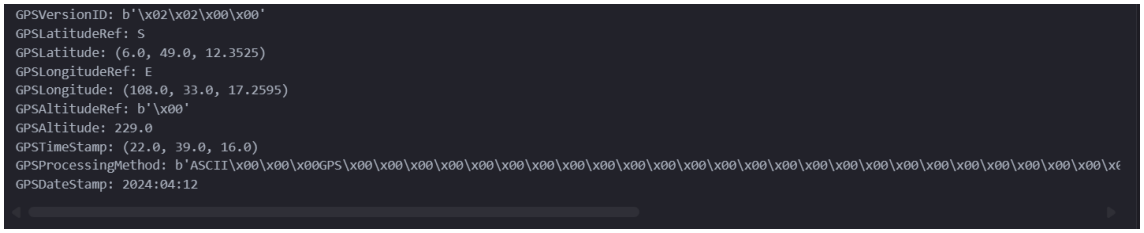
Mencetak nama tag dan nilainya dengan f-string. Bagian `{tag_name}` dan `{value}` diganti dengan isi variabel pada setiap putaran perulangan.

Baris 19–20

```
else:
    print("Data koordinat GPS tidak ditemukan.")
```

Jika `gps_data` kosong, program hanya menampilkan pesan bahwa data koordinat GPS tidak ditemukan. Pada foto ini cabang `else` tidak dijalankan karena data GPS tersedia.

4. Hasil program


GPSVersionID: b'\x02\x02\x00\x00'
GPSLatitudeRef: S
GPSLatitude: (6.0, 49.0, 12.3525)
GPSLongitudeRef: E
GPSLongitude: (108.0, 33.0, 17.2595)
GPSAltitudeRef: b'\x00'
GPSAltitude: 229.0
GPSTimeStamp: (22.0, 39.0, 16.0)
GPSProcessingMethod: b'ASCII\x00\x00\x00GPS\x00\x00\x00

2. Screenshot kode program

```python
# ========================================
# 6. MENGUBAH KOORDINAT GPS MENJADI DESIMAL
# ========================================

# Mengubah koordinat derajat, menit, detik menjadi desimal
def ke_desimal(dms, arah):
    derajat = float(dms[0])
    menit = float(dms[1])
    detik = float(dms[2])

    hasil = derajat + menit / 60 + detik / 3600

    # Menentukan tanda koordinat berdasarkan arah
    if arah in ["S", "W"]:
        hasil = -hasil

    return hasil


# Membaca metadata GPS dari gambar
gps = gambar.getexif().get_ifd(ExifTags.IFD.GPSInfo)

# Memeriksa kelengkapan koordinat GPS
if gps and 2 in gps and 4 in gps:
    lintang = ke_desimal(gps[2], gps.get(1, "N"))
    bujur = ke_desimal(gps[4], gps.get(3, "E"))

    # Membuat tautan Google Maps
    link_maps = (
        f"https://www.google.com/maps?q={lintang},{bujur}"
    )
```

Gambar 10. Kode program nomor 6 bagian 1 (fungsi konversi dan tautan Google Maps)

```python
# Menyusun koordinat dalam bentuk tabel
tabel_gps = pd.DataFrame({
    "Informasi": [
        "Latitude (Lintang)",
        "Longitude (Bujur)",
        "Koordinat Desimal"
    ],
    "Nilai": [
        f"{lintang:.6f}°",
        f"{bujur:.6f}°",
        f"{lintang:.6f}, {bujur:.6f}"
    ]
})

# Menampilkan tabel koordinat
display(tabel_gps)

# Menampilkan tautan lokasi di Google Maps
from IPython.display import HTML
display(HTML(
    f'<a href="{link_maps}" target="_blank">'
    'Buka lokasi di Google Maps</a>'
))
```

Gambar 11. Kode program nomor 6 bagian 2 (tabel koordinat dan tautan lokasi)

```python
    # Membuat peta interaktif jika Folium tersedia
    try:
        import folium

        peta = folium.Map(
            location=[lintang, bujur],
            zoom_start=16
        )

        # Menambahkan penanda lokasi pada peta
        folium.Marker(
            [lintang, bujur],
            tooltip="Lokasi dari metadata foto",
            popup=f"Koordinat: {lintang:.6f}, {bujur:.6f}"
        ).add_to(peta)

        # Menampilkan peta
        display(peta)

    except ImportError:
        print(
            "Peta interaktif belum tersedia. "
            "Gunakan tautan Google Maps di atas."
        )

else:
    print("Koordinat GPS tidak lengkap atau tidak tersedia.")
```

✓ 0.0s

Gambar 12. Kode program nomor 6 bagian 3 (peta interaktif dan penanganan kondisi lain)

3. Penjelasan setiap baris kode

Baris 1–3

```
# ====...
# 6. MENGUBAH KOORDINAT GPS MENJADI DESIMAL
# ====...
```

Komentar penanda tahap nomor 6.

Baris 5–6

```
# Mengubah koordinat derajat, menit, detik menjadi desimal
def ke_desimal(dms, arah):
```

Mendefinisikan fungsi bernama `ke_desimal` dengan dua parameter. `dms` berisi tiga angka (derajat, menit, detik) dan `arah` berisi huruf arah mata angin (N, S, E, atau W).

Baris 7–9

```
    derajat = float(dms[0])
    menit = float(dms[1])
    detik = float(dms[2])
```

Mengambil elemen indeks 0, 1, dan 2 dari `dms`, yaitu derajat, menit, dan detik. Setiap nilai diubah dengan `float()` menjadi bilangan desimal agar dapat dihitung.

Baris 11

```
    hasil = derajat + menit / 60 + detik / 3600
```

Rumus konversi ke derajat desimal. Satu derajat terdiri atas 60 menit dan satu menit terdiri atas 60 detik, sehingga menit dibagi 60 dan detik dibagi 3600 (60 × 60) sebelum dijumlahkan dengan derajat.

Baris 13–15

```
    # Menentukan tanda koordinat berdasarkan arah
    if arah in ["S", "W"]:
        hasil = -hasil
```

Jika arahnya Selatan (S) atau Barat (W), nilai dibuat negatif. Arah Utara (N) dan Timur (E) tetap bernilai positif.

Baris 17

```
    return hasil
```

Mengembalikan nilai desimal sebagai hasil fungsi.

Baris 20–21

```
# Membaca metadata GPS dari gambar
gps = gambar.getexif().get_ifd(ExifTags.IFD.GPSInfo)
```

Mengambil metadata GPS langsung dari gambar dalam satu baris, menggabungkan dua langkah pada nomor 5 (`getexif()` dan `get_ifd()`). Hasilnya disimpan pada variabel `gps`.

Baris 23–24

```
# Memeriksa kelengkapan koordinat GPS
if gps and 2 in gps and 4 in gps:
```

Memeriksa tiga syarat sekaligus: data GPS tersedia, tag dengan ID 2 (`GPSLatitude`) ada, dan tag dengan ID 4 (`GPSLongitude`) ada. Blok di bawahnya hanya dijalankan jika ketiganya terpenuhi.

Baris 25

```
    lintang = ke_desimal(gps[2], gps.get(1, "N"))
```

Memanggil fungsi `ke_desimal` untuk lintang. `gps[2]` adalah nilai `GPSLatitude` (derajat, menit, detik), sedangkan `gps.get(1, "N")` mengambil arahnya dari tag ID 1 (`GPSLatitudeRef`), dengan nilai cadangan "N" jika tag tidak ada. Hasilnya disimpan pada `lintang`.

Baris 26

```
    bujur = ke_desimal(gps[4], gps.get(3, "E"))
```

Memanggil fungsi yang sama untuk bujur. `gps[4]` adalah `GPSLongitude`, dan `gps.get(3, "E")` mengambil arah dari tag ID 3 (`GPSLongitudeRef`), dengan nilai cadangan "E". Hasilnya disimpan pada `bujur`.

Baris 28–31

```
    # Membuat tautan Google Maps
    link_maps = (
        f"https://www.google.com/maps?q={lintang},{bujur}"
    )
```

Menyusun tautan Google Maps dengan f-string. Nilai `lintang` dan `bujur` disisipkan setelah `?q=` sehingga Google Maps menandai titik koordinat tersebut. Tanda kurung hanya membungkus string agar penulisannya rapi. Hasilnya disimpan pada `link_maps`.

Baris 33–45

```
# Menyusun koordinat dalam bentuk tabel
tabel_gps = pd.DataFrame({
    "Informasi": [
        "Latitude (Lintang)",
        "Longitude (Bujur)",
        "Koordinat Desimal"
    ],
    "Nilai": [
        f"{lintang:.6f}°",
        f"{bujur:.6f}°",
        f"{lintang:.6f}, {bujur:.6f}"
    ]
})
```

Membuat tabel Pandas dari sebuah kamus yang memiliki dua kolom. Kolom `Informasi` berisi tiga label, dan kolom `Nilai` berisi tiga nilai yang sesuai. Format `{lintang:.6f}` menampilkan angka dengan 6 angka di belakang koma, lalu tanda ° ditambahkan. Baris terakhir pada kolom `Nilai` menggabungkan lintang dan bujur yang dipisahkan koma. Tabel disimpan pada `tabel_gps`.

Baris 47–48

```
# Menampilkan tabel koordinat
display(tabel_gps)
```

`display()` menampilkan tabel pada output notebook. Fungsi ini dipakai karena tabel bukan baris terakhir sel, sehingga tidak tampil otomatis.

Baris 50–51

```
# Menampilkan tautan lokasi di Google Maps
from IPython.display import HTML
```

Mengimpor `HTML` dari `IPython.display`, yaitu komponen untuk menampilkan teks HTML pada output notebook.

Baris 52–55

```
display(HTML(
    f'<a href="{link_maps}" target="_blank">'
    'Buka lokasi di Google Maps</a>'
))
```

Menampilkan tautan berbentuk HTML. Tag `<a href=...>` membuat tautan ke alamat `link_maps`, `target="_blank"` membuatnya terbuka di tab baru, dan teks yang tampil adalah "Buka lokasi di Google Maps". Dua string pada baris 53 dan 54 otomatis digabung oleh Python.

Baris 57–59

```
# Membuat peta interaktif jika Folium tersedia
try:
    import folium
```

Mencoba mengimpor library Folium yang dipakai untuk membuat peta interaktif. Perintah ini dibungkus `try` agar program tidak berhenti dengan kesalahan jika Folium belum terpasang.

Baris 61–64

```
peta = folium.Map(
    location=[lintang, bujur],
    zoom_start=16
)
```

Membuat peta Folium yang berpusat pada koordinat foto (`location`). `zoom_start=16` mengatur tingkat perbesaran awal peta; semakin besar angkanya, semakin dekat tampilan peta. Peta disimpan pada `peta`.

Baris 66–71

```
# Menambahkan penanda lokasi pada peta
folium.Marker(
    [lintang, bujur],
    tooltip="Lokasi dari metadata foto",
    popup=f"Koordinat: {lintang:.6f}, {bujur:.6f}"
).add_to(peta)
```

Membuat penanda (marker) pada titik koordinat foto. `tooltip` adalah teks yang muncul saat kursor diarahkan ke penanda, sedangkan `popup` adalah teks berisi koordinat yang muncul saat penanda diklik. `.add_to(peta)` menempelkan penanda ke peta.

Baris 73–74

```
# Menampilkan peta
display(peta)
```

Menampilkan peta interaktif pada output notebook.

Baris 76–80

```
except ImportError:
    print(
        "Peta interaktif belum tersedia. "
        "Gunakan tautan Google Maps di atas."
    )
```

Dijalankan jika `import folium` gagal karena library belum terpasang. Program mencetak pesan bahwa peta interaktif belum tersedia dan mengarahkan pengguna ke tautan Google Maps. Dua string pada baris 78 dan 79 otomatis digabung menjadi satu kalimat.

Baris 82–83

```
else:
    print("Koordinat GPS tidak lengkap atau tidak tersedia.")
```

Cabang `else` dari pemeriksaan pada baris 24. Pesan ini dicetak jika data GPS kosong atau tag lintang dan bujur tidak lengkap. Pada foto ini cabang tersebut tidak dijalankan.

4. Hasil program



	Informasi	Nilai
0	Latitude (Lintang)	-6.820098°
1	Longitude (Bujur)	108.554794°
2	Koordinat Desimal	-6.820098, 108.554794

Buka lokasi di Google Maps

Peta interaktif belum tersedia. Gunakan tautan Google Maps di atas.

Gambar 13. Hasil konversi koordinat GPS pada nomor 6

5. Analisis hasil

Fungsi `ke_desimal` menghasilkan lintang -6.820098° dan bujur 108.554794°. Hasil ini dapat diperiksa dengan perhitungan manual. Untuk lintang, 6 + 49/60 + 12,3525/3600 = 6,820098, dan karena arahnya S (Selatan) nilainya menjadi negatif. Untuk bujur, 108 + 33/60 + 17,2595/3600 = 108,554794, dan arahnya E (Timur) sehingga nilainya tetap positif. Tabel menampilkan lintang, bujur, dan gabungan keduanya sebagai koordinat desimal -6.820098, 108.554794.

Tautan "Buka lokasi di Google Maps" berhasil dibuat dari koordinat tersebut. Adapun peta interaktif tidak tampil: pesan "Peta interaktif belum tersedia. Gunakan tautan Google Maps di atas." muncul, yang berarti `import folium` gagal sehingga blok `except ImportError` dijalankan. Karena itu, lokasi pada notebook ini dilihat melalui tautan Google Maps. Variabel `lintang`, `bujur`, dan `link_maps` dipakai kembali pada ringkasan di nomor 14.

Codingan Nomor 7: Membuktikan Citra Tersusun dari Pixel

1. Tujuan codingan

Mengubah gambar menjadi array NumPy, kemudian memeriksa dimensi, jumlah pixel, tipe data, dan kedalaman bit untuk membuktikan bahwa citra tersusun dari pixel bernilai angka.

2. Screenshot kode program

```
# ========================================
# 7. MEMBUKTIKAN CITRA TERSUSUN DARI PIXEL
# ========================================

# Mengubah gambar menjadi array angka
array_gambar = np.array(gambar)

# Menampilkan bentuk dan dimensi array
print("Bentuk array       :", array_gambar.shape)
print("Tinggi gambar      :", array_gambar.shape[0], "pixel")
print("Lebar gambar       :", array_gambar.shape[1], "pixel")
print("Jumlah kanal warna :", array_gambar.shape[2])

# Menghitung jumlah seluruh pixel
jumlah_pixel = array_gambar.shape[0] * array_gambar.shape[1]
print("Jumlah pixel       :", f"{jumlah_pixel:,}".replace(",", "."))

# Mengetahui tipe data setiap nilai pixel
print("Tipe data pixel    :", array_gambar.dtype)

# Mengetahui kedalaman bit setiap kanal warna
kedalaman_bit = array_gambar.dtype.itemsize * 8
print("Kedalaman bit      :", kedalaman_bit, "bit per kanal")

# Menghitung jumlah bit untuk satu pixel RGB
bit_per_pixel = kedalaman_bit * array_gambar.shape[2]
print("Bit per pixel RGB  :", bit_per_pixel, "bit")
✓ 0.0s
```

Gambar 14. Kode program nomor 7 (membuktikan citra tersusun dari pixel)

3. Penjelasan setiap baris kode

Baris 1–3

```
# ====...
# 7. MEMBUKTIKAN CITRA TERSUSUN DARI PIXEL
# ====...
```

Komentar penanda tahap nomor 7.

Baris 5–6

```
# Mengubah gambar menjadi array angka
array_gambar = np.array(gambar)
```

`np.array()` mengubah objek gambar menjadi array NumPy yang berisi nilai pixel. Hasilnya disimpan pada `array_gambar`, variabel penting yang dipakai sampai nomor 14.

Baris 8–9

```
# Menampilkan bentuk dan dimensi array
print("Bentuk array       :", array_gambar.shape)
```

`shape` menampilkan bentuk array dalam urutan (tinggi, lebar, kanal). Hasilnya (2448, 3264, 3). Perhatikan bahwa urutan ini berbeda dengan `gambar.size` pada nomor 4 yang berurutan (lebar, tinggi): array menyimpan baris terlebih dahulu, sehingga tinggi muncul lebih dulu.

Baris 10

```
print("Tinggi gambar      :", array_gambar.shape[0], "pixel")
```

Indeks `[0]` mengambil elemen pertama dari `shape`, yaitu tinggi gambar atau jumlah baris pixel: 2448.

Baris 11

```
print("Lebar gambar       :", array_gambar.shape[1], "pixel")
```

Indeks `[1]` mengambil jumlah kolom pixel, yaitu lebar gambar: 3264.

Baris 12

```
print("Jumlah kanal warna :", array_gambar.shape[2])
```

Indeks `[2]` mengambil jumlah kanal warna pada setiap pixel: 3 (R, G, B).

Baris 14–16

```
# Menghitung jumlah seluruh pixel
jumlah_pixel = array_gambar.shape[0] * array_gambar.shape[1]
print("Jumlah pixel       :", f"{jumlah_pixel:,}".replace(",", "."))
```

Jumlah pixel dihitung dari tinggi × lebar, yaitu 2448 × 3264. Kanal tidak ikut dikalikan karena tiga kanal merupakan komponen warna dari satu pixel yang sama. Pada `print()`, `f"{jumlah_pixel:,}"` memformat angka dengan pemisah ribuan berupa koma, lalu `.replace(",", ".")` mengganti koma dengan titik agar sesuai penulisan Indonesia, sehingga tampil 7.990.272.

Baris 18–19

```
# Mengetahui tipe data setiap nilai pixel
print("Tipe data pixel    :", array_gambar.dtype)
```

`dtype` menunjukkan tipe data yang dipakai untuk menyimpan setiap nilai pada array. Hasilnya `uint8`, yaitu bilangan bulat tanpa tanda (unsigned integer) 8 bit dengan rentang nilai 0–255.

Baris 21–23

```
# Mengetahui kedalaman bit setiap kanal warna
kedalaman_bit = array_gambar.dtype.itemsize * 8
print("Kedalaman bit      :", kedalaman_bit, "bit per kanal")
```

`dtype.itemsize` adalah ukuran satu nilai dalam satuan byte (untuk `uint8` bernilai 1). Dikalikan 8 karena 1 byte sama dengan 8 bit, sehingga kedalaman bit adalah 8 bit per kanal.

Baris 25–27

```
# Menghitung jumlah bit untuk satu pixel RGB
bit_per_pixel = kedalaman_bit * array_gambar.shape[2]
print("Bit per pixel RGB  :", bit_per_pixel, "bit")
```

Jumlah bit untuk satu pixel RGB diperoleh dari kedalaman bit dikalikan jumlah kanal, yaitu 8 × 3 = 24 bit per pixel.

4. Hasil program

```
Bentuk array       : (2448, 3264, 3)
Tinggi gambar      : 2448 pixel
Lebar gambar       : 3264 pixel
Jumlah kanal warna : 3
Jumlah pixel       : 7.990.272
Tipe data pixel    : uint8
Kedalaman bit      : 8 bit per kanal
Bit per pixel RGB  : 24 bit
```

Gambar 15. Hasil program nomor 7

5. Analisis hasil

Foto yang tampak sebagai gambar utuh ternyata dapat dibaca sebagai array tiga dimensi berbentuk (2448, 3264, 3): 2448 baris, 3264 kolom, dan 3 nilai pada setiap posisi. Total terdapat 2448 × 3264 = 7.990.272 pixel. Setiap nilai kanal disimpan sebagai `uint8` (8 bit, rentang 0–255), sehingga satu pixel RGB memerlukan 3 × 8 = 24 bit. Hasil ini membuktikan bahwa citra digital memiliki struktur data numerik yang tersusun dari pixel dan nilai warna.

Codingan Nomor 8: Melihat Nilai Pixel dalam Bentuk Angka

1. Tujuan codingan

Mengambil sebagian kecil array, yaitu area 3 × 3 pixel pertama, untuk melihat nilai RGB pixel secara langsung dalam bentuk angka.

2. Screenshot kode program

```
# ========================================
# 8. MELIHAT NILAI PIXEL DALAM BENTUK ANGKA
# ========================================

pixel = array_gambar[0:3, 0:3]

print("Nilai pixel pada bagian 3 x 3 pertama:")
print(pixel)
```

✓ 0.0s

Gambar 16. Kode program nomor 8 (nilai pixel area 3 × 3)

3. Penjelasan setiap baris kode

Baris 1–3

```
# ====...
# 8. MELIHAT NILAI PIXEL DALAM BENTUK ANGKA
# ====...
```

Komentar penanda tahap nomor 8.

Baris 5

```
pixel = array_gambar[0:3, 0:3]
```

Mengambil potongan (slicing) dari `array_gambar`. Angka pertama `0:3` memilih baris indeks 0 sampai 2, dan angka kedua `0:3` memilih kolom indeks 0 sampai 2 (batas akhir 3 tidak ikut diambil). Karena dimensi ketiga (kanal) tidak disebutkan, ketiga nilai R, G, B ikut terambil. Hasilnya 3 baris × 3 kolom = 9 pixel di sudut kiri atas gambar, disimpan pada variabel `pixel`.

Baris 7

```
print("Nilai pixel pada bagian 3 × 3 pertama:")
```

Mencetak keterangan agar output mudah dipahami.

Baris 8

```
print(pixel)
```

Mencetak isi potongan array berupa nilai angka.

4. Hasil program

```
Nilai pixel pada bagian 3 × 3 pertama:
[[[151 147 136]
  [145 141 130]
  [149 147 135]]

 [[152 148 137]
  [147 143 132]
  [147 145 133]]

 [[147 143 132]
  [145 141 130]
  [146 144 132]]]
```

Gambar 17. Hasil program nomor 8

5. Analisis hasil

Output berupa tiga kelompok baris, dan setiap pixel ditulis sebagai tiga angka. Contohnya, pixel pertama bernilai `[151 147 136]`, yang berarti R = 151, G = 147, dan B = 136, masing-masing dalam rentang 0–255. Nilai-nilai pada sembilan pixel ini saling berdekatan (berkisar antara 130 sampai 152), yang wajar karena pixel bertetangga

biasanya memiliki warna yang mirip. Hasil ini menunjukkan secara langsung bahwa pixel pada foto adalah angka.

Codingan Nomor 9: Menentukan Beberapa Koordinat Pixel yang Akan Dianalisis

1. Tujuan codingan

Menentukan lima koordinat pixel pada citra, mengambil nilai RGB dari setiap koordinat tersebut, menyusunnya dalam tabel Pandas, dan menampilkan warna setiap pixel secara visual menggunakan Matplotlib sehingga nilai angka RGB lebih mudah dipahami.

2. Screenshot kode program

```
# ========================================
# 9. MENENTUKAN BEBERAPA KOORDINAT PIXEL YANG AKAN DIANALISIS
# ========================================

# Menentukan koordinat pixel yang akan dianalisis
koordinat_pixel = [
    (0, 0),
    (0, 100),
    (100, 100),
    (200, 200),
    (500, 500)
]

# Menyiapkan tempat untuk menyimpan hasil analisis
data_pixel = []

# Mengambil nilai RGB dari setiap koordinat pixel
for baris, kolom in koordinat_pixel:
    r, g, b = array_gambar[baris, kolom]

    # Menyimpan koordinat dan nilai RGB
    data_pixel.append({
        "Koordinat (Baris, Kolom)": f"({baris}, {kolom})",
        "Red (R)": int(r),
        "Green (G)": int(g),
        "Blue (B)": int(b),
        "Warna RGB": f"({r}, {g}, {b})"
    })

# Mengubah hasil analisis menjadi tabel
tabel_pixel = pd.DataFrame(data_pixel)

# Menampilkan tabel nilai RGB
display(tabel_pixel)

# Menampilkan warna dari setiap koordinat pixel
plt.figure(figsize=(8, 2))
```

Gambar 18. Kode program nomor 9 bagian 1 (penentuan koordinat, pengambilan nilai RGB, dan tabel)



```
# Menampilkan warna dari setiap koordinat pixel
plt.figure(figsize=(8, 2))

for i, (baris, kolom) in enumerate(koordinat_pixel):

    # Mengambil kembali nilai RGB pixel
    r, g, b = array_gambar[baris, kolom]

    # Menampilkan warna pixel
    plt.subplot(1, len(koordinat_pixel), i + 1)
    plt.imshow([[[r, g, b]]])
    plt.title(f"({baris}, {kolom})")
    plt.axis("off")

# Memberikan judul dan merapikan tampilan
plt.suptitle("Warna Pixel Berdasarkan Koordinat")
plt.tight_layout()
plt.show()

✓ 0.3s
```

Gambar 19. Kode program nomor 9 bagian 2 (visualisasi warna pixel)

3. Penjelasan setiap baris kode

Baris 1–3

```
# ====...
# 9. MENENTUKAN BEBERAPA KOORDINAT PIXEL YANG AKAN DIANALISIS
# ====...
```

Komentar penanda tahap nomor 9.

Baris 5–12

```
# Menentukan koordinat pixel yang akan dianalisis
koordinat_pixel = [
    (0, 0),
    (0, 100),
    (100, 100),
    (200, 200),
    (500, 500)
]
```

Membuat daftar (list) bernama `koordinat_pixel` yang berisi lima koordinat pixel. Setiap koordinat ditulis sebagai pasangan (baris, kolom), yaitu (0, 0), (0, 100), (100, 100), (200, 200), dan (500, 500). Urutan ini mengikuti urutan indeks array pada nomor 7, yaitu baris terlebih dahulu kemudian kolom. Pixel (0, 0) berada di sudut kiri atas gambar, dan keempat koordinat lainnya berada dalam batas ukuran gambar (2448 baris dan 3264 kolom) sehingga dapat dibaca.

Baris 14–15

```
# Menyiapkan tempat untuk menyimpan hasil analisis
data_pixel = []
```

Membuat daftar kosong bernama `data_pixel` untuk menampung hasil pengambilan koordinat dan nilai RGB. Setiap pixel nantinya ditambahkan ke daftar ini sebagai satu kamus (dictionary).

Baris 17–18

```
# Mengambil nilai RGB dari setiap koordinat pixel
for baris, kolom in koordinat_pixel:
```

Perulangan yang memproses setiap koordinat pada `koordinat_pixel`. Pada setiap putaran, pasangan (baris, kolom) dipecah ke dua variabel, sehingga terdapat lima putaran sesuai jumlah koordinat.

Baris 19

```
    r, g, b = array_gambar[baris, kolom]
```

Mengambil satu pixel dari `array_gambar` berdasarkan indeks baris dan kolom. Karena indeks kanal tidak disebutkan, hasilnya berupa tiga angka yang langsung dipecah ke variabel `r` (Red), `g` (Green), dan `b` (Blue), sesuai urutan kanal pada mode RGB.

Baris 21–28

```
    # Menyimpan koordinat dan nilai RGB
    data_pixel.append({
        "Koordinat (Baris, Kolom)": f"({baris}, {kolom})",
        "Red (R)": int(r),
        "Green (G)": int(g),
        "Blue (B)": int(b),
        "Warna RGB": f"({r}, {g}, {b})"
    })
```

Menambahkan satu kamus berisi lima data ke `data_pixel`. Kunci `"Koordinat (Baris, Kolom)"` menyimpan teks koordinat yang disusun dengan f-string. Tiga kunci berikutnya menyimpan nilai R, G, dan B; fungsi `int()` mengubah nilai `uint8` NumPy menjadi bilangan bulat biasa. Kunci `"Warna RGB"` menggabungkan ketiga nilai menjadi satu teks, misalnya (151, 147, 136). Kunci-kunci ini nantinya menjadi nama kolom tabel.

Baris 30–31

```
# Mengubah hasil analisis menjadi tabel
tabel_pixel = pd.DataFrame(data_pixel)
```

`pd.DataFrame()` mengubah daftar kamus menjadi tabel. Setiap kamus menjadi satu baris tabel dan kunci kamus menjadi nama kolom. Tabel disimpan pada `tabel_pixel`.

Baris 33–34

```
# Menampilkan tabel nilai RGB
display(tabel_pixel)
```

`display()` menampilkan tabel yang berisi koordinat dan nilai RGB setiap pixel pada output notebook. Fungsi ini dipakai karena tabel bukan baris terakhir sel.

Baris 36–37

```
# Menampilkan warna dari setiap koordinat pixel
plt.figure(figsize=(8, 2))
```

Menyiapkan area gambar Matplotlib berukuran 8 × 2 inci. Bentuknya lebar dan pendek karena area ini dipakai untuk menampung lima kotak warna yang berjajar ke samping.

Baris 39

```
for i, (baris, kolom) in enumerate(koordinat_pixel):
```

Perulangan kedua untuk setiap koordinat. `enumerate()` memberikan nomor urut `i` (mulai dari 0) sekaligus pasangan (baris, kolom). Nomor `i` dipakai untuk menentukan posisi tiap kotak warna.

Baris 41–42

```
    # Mengambil kembali nilai RGB pixel
    r, g, b = array_gambar[baris, kolom]
```

Mengambil kembali nilai RGB pixel dengan cara yang sama seperti pada baris 19, karena variabel `r`, `g`, dan `b` perlu diperbarui pada setiap putaran perulangan visualisasi.

Baris 44–46

```
    # Menampilkan warna pixel
    plt.subplot(1, len(koordinat_pixel), i + 1)
    plt.imshow([[[r, g, b]]])
```

`plt.subplot(1, len(koordinat_pixel), i + 1)` membagi area gambar menjadi 1 baris dan 5 kolom (hasil `len(koordinat_pixel)`), lalu memilih panel ke-`i + 1` (penomoran panel dimulai dari 1). `plt.imshow([[[r, g, b]]])` menampilkan data berukuran 1 × 1 pixel dengan tiga nilai RGB, sehingga yang tampil adalah satu blok warna dari pixel yang sedang dianalisis.

Baris 47

```
    plt.title(f"({baris}, {kolom})")
```

Memberi judul di atas setiap blok warna berupa koordinat pixel, sehingga posisi pixel mudah dikenali.

Baris 48

```
    plt.axis("off")
```

Menyembunyikan sumbu agar visualisasi hanya berfokus pada warna pixel.

Baris 50–52

```
# Memberikan judul dan merapikan tampilan
plt.suptitle("Warna Pixel Berdasarkan Koordinat")
plt.tight_layout()
```

`plt.suptitle()` memberi judul utama untuk seluruh area gambar, sedangkan `plt.tight_layout()` mengatur jarak antarpanel agar judul dan blok warna tersusun rapi dan tidak saling menimpa.

Baris 53

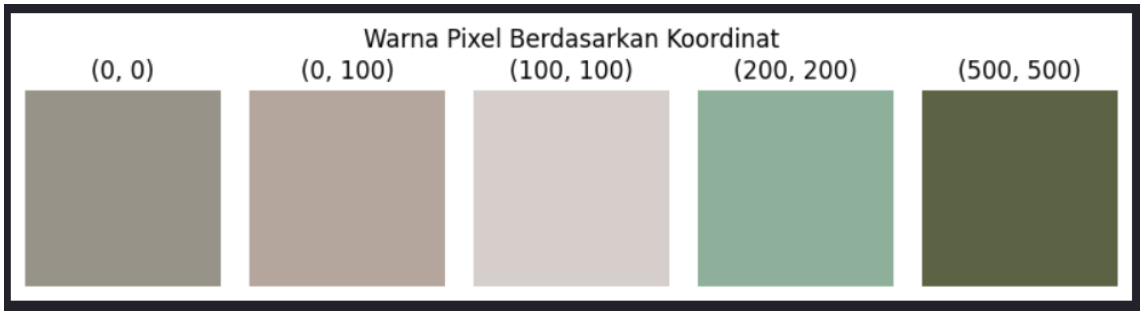
```
plt.show()
```

Menampilkan hasil visualisasi warna pixel.

4. Hasil program

	Koordinat (Baris, Kolom)	Red (R)	Green (G)	Blue (B)	Warna RGB
0	(0, 0)	151	147	136	(151, 147, 136)
1	(0, 100)	180	166	157	(180, 166, 157)
2	(100, 100)	214	206	203	(214, 206, 203)
3	(200, 200)	142	175	154	(142, 175, 154)
4	(500, 500)	92	99	68	(92, 99, 68)

Gambar 20. Tabel nilai RGB lima koordinat pixel pada nomor 9



Gambar 21. Visualisasi warna lima pixel pada nomor 9

5. Analisis hasil

Lima koordinat pixel berhasil dibaca dan menghasilkan nilai RGB yang berbeda-beda: (0, 0) bernilai (151, 147, 136), (0, 100) bernilai (180, 166, 157), (100, 100) bernilai (214, 206, 203), (200, 200) bernilai (142, 175, 154), dan (500, 500) bernilai (92, 99, 68). Nilai pixel (0, 0) sama dengan pixel pertama pada hasil nomor 8, yaitu [151 147 136], sehingga pembacaan melalui koordinat konsisten dengan potongan array sebelumnya.

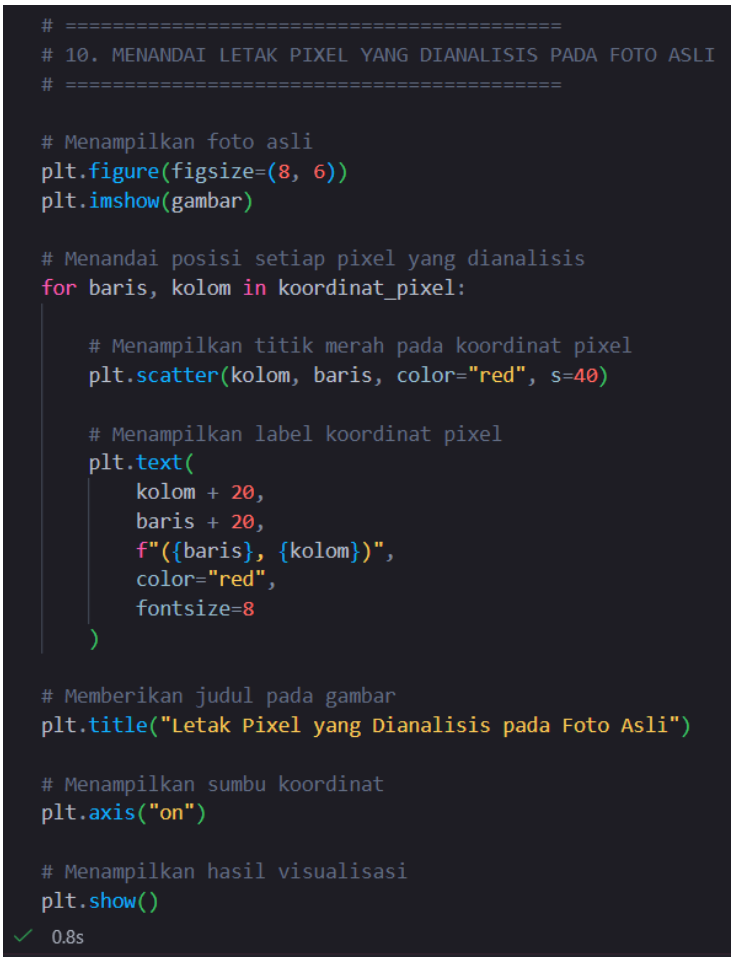
Perbedaan nilai tersebut tampak pada visualisasi warna. Pixel (100, 100) memiliki nilai R, G, dan B yang tertinggi di antara kelima pixel dan ketiganya berdekatan, sehingga warnanya tampak terang keabu-abuan. Pixel (500, 500) memiliki nilai terendah pada ketiga kanal sehingga warnanya paling gelap, sedangkan pixel (200, 200) memiliki nilai G yang lebih tinggi daripada R dan B sehingga warnanya tampak kehijauan. Hasil ini membuktikan bahwa setiap pixel memiliki posisi dan komposisi warna tertentu, dan bahwa warna yang terlihat pada gambar ditentukan oleh tiga angka RGB.

Codingan Nomor 10: Menandai Letak Pixel yang Dianalisis pada Foto Asli

1. Tujuan codingan

Menampilkan foto asli dan menandai posisi lima pixel yang ditentukan pada nomor 9 menggunakan titik merah dan label koordinat, sehingga letak setiap pixel dapat dilihat langsung pada gambar.

2. Screenshot kode program



```
# ========================================
# 10. MENANDAI LETAK PIXEL YANG DIANALISIS PADA FOTO ASLI
# ========================================

# Menampilkan foto asli
plt.figure(figsize=(8, 6))
plt.imshow(gambar)

# Menandai posisi setiap pixel yang dianalisis
for baris, kolom in koordinat_pixel:

    # Menampilkan titik merah pada koordinat pixel
    plt.scatter(kolom, baris, color="red", s=40)

    # Menampilkan label koordinat pixel
    plt.text(
        kolom + 20,
        baris + 20,
        f"({baris}, {kolom})",
        color="red",
        fontsize=8
    )

# Memberikan judul pada gambar
plt.title("Letak Pixel yang Dianalisis pada Foto Asli")

# Menampilkan sumbu koordinat
plt.axis("on")

# Menampilkan hasil visualisasi
plt.show()
```

0.8s

Gambar 22. Kode program nomor 10 (menandai letak pixel pada foto asli)

3. Penjelasan setiap baris kode

Baris 1–3

```
# ====...
# 10. MENANDAI LETAK PIXEL YANG DIANALISIS PADA FOTO ASLI
# ====...
```

Komentar penanda tahap nomor 10.

Baris 5–7

```
# Menampilkan foto asli
plt.figure(figsize=(8, 6))
plt.imshow(gambar)
```

`plt.figure(figsize=(8, 6))` membuat area visualisasi berukuran 8 × 6 inci. `plt.imshow(gambar)` menampilkan foto asli yang tersimpan pada variabel `gambar` (nomor 2). Pada tampilan ini, sumbu horizontal menunjukkan nomor kolom pixel dan sumbu vertikal menunjukkan nomor baris pixel, dengan baris 0 berada di bagian atas gambar.

Baris 9–10

```
# Menandai posisi setiap pixel yang dianalisis
for baris, kolom in koordinat_pixel:
```

Perulangan yang mengambil setiap koordinat pada `koordinat_pixel` (variabel dari nomor 9). Terdapat lima putaran, sehingga kelima pixel yang sama dengan nomor 9 ditandai satu per satu.

Baris 12–13

```
    # Menampilkan titik merah pada koordinat pixel
    plt.scatter(kolom, baris, color="red", s=40)
```

`plt.scatter()` menggambar sebuah titik pada gambar. Argumen pertama adalah posisi horizontal (x), yaitu `kolom`, dan argumen kedua adalah posisi vertikal (y), yaitu `baris`. Urutannya terbalik dibandingkan penulisan koordinat (baris, kolom) karena Matplotlib menerima nilai x terlebih dahulu. `color="red"` mengatur warna titik menjadi merah dan `s=40` mengatur ukurannya.

Baris 15–22

```
    # Menampilkan label koordinat pixel
    plt.text(
        kolom + 20,
        baris + 20,
        f"({baris}, {kolom})",
        color="red",
        fontsize=8
    )
```

`plt.text()` menuliskan teks pada gambar. Dua argumen pertama, `kolom + 20` dan `baris + 20`, menggeser posisi teks 20 pixel dari titik sehingga label tidak menutupi titik merah. Argumen ketiga, `f"({baris}, {kolom})"`, adalah isi label berupa koordinat pixel. `color="red"` mengatur warna tulisan dan `fontsize=8` mengatur ukuran hurufnya.

Baris 24–25

```
# Memberikan judul pada gambar
plt.title("Letak Pixel yang Dianalisis pada Foto Asli")
```

Memberi judul pada visualisasi.

Baris 27–28

```
# Menampilkan sumbu koordinat
plt.axis("on")
```

Menampilkan sumbu koordinat pada gambar. Pada nomor 3 dan nomor 12 sumbu disembunyikan dengan `"off"`, sedangkan di sini sumbu sengaja ditampilkan agar posisi pixel dapat dibaca dari angka pada sumbu.

Baris 30–31

```
# Menampilkan hasil visualisasi
plt.show()
```

Menampilkan hasil akhir berupa foto asli dengan titik merah dan label koordinat.

4. Hasil program



Gambar 23. Letak lima pixel yang dianalisis pada foto asli (nomor 10)

5. Analisis hasil

Hasil visualisasi menunjukkan lima titik merah beserta label koordinatnya pada foto asli. Sumbu horizontal pada gambar mencakup kolom 0 sampai sekitar 3264 dan sumbu vertikal mencakup baris 0 sampai sekitar 2448, sesuai ukuran gambar pada nomor 4. Kelima titik berada di bagian kiri atas foto karena nilai baris dan kolomnya kecil (maksimal 500 dari 2448 baris dan 3264 kolom). Titik (0, 0) berada di sudut kiri atas, sedangkan titik (0, 100) berada di tepi atas pada jarak 100 pixel dari titik tersebut sehingga

kedua label tampak saling berdekatan dan sebagian bertumpuk. Titik (100, 100), (200, 200), dan (500, 500) bergerak semakin ke kanan dan ke bawah sesuai nilai koordinatnya.

Hubungan dengan nomor 9: nomor 9 menunjukkan nilai RGB dan warna dari lima pixel terpilih, sedangkan nomor 10 menunjukkan letak kelima pixel tersebut pada foto asli. Kedua tahap memakai variabel `koordinat_pixel` yang sama, sehingga setiap titik merah pada nomor 10 dapat dicocokkan dengan satu baris tabel dan satu blok warna pada nomor 9. Dengan demikian terbukti bahwa setiap pixel memiliki posisi tertentu (baris dan kolom) sekaligus nilai warna tertentu (R, G, dan B) di dalam citra digital.

Codingan Nomor 11: Mengubah Gambar Menjadi Matriks Angka

1. Tujuan codingan

Menampilkan sebagian citra berukuran 5 × 5 pixel sebagai contoh matriks angka. Hanya sebagian yang ditampilkan karena seluruh gambar terlalu besar untuk dicetak.

2. Screenshot kode program

```
# ========================================
# 11. MENGUBAH GAMBAR MENJADI MATRIKS ANGKA
# ========================================

matriks_pixel = array_gambar[0:5, 0:5]

print("Matriks pixel 5 x 5:")
print(matriks_pixel)
```

✓ 0.0s

Gambar 24. Kode program nomor 11 (matriks pixel 5 × 5)

3. Penjelasan setiap baris kode

Baris 1–3

```
# ====...
# 11. MENGUBAH GAMBAR MENJADI MATRIKS ANGKA
# ====...
```

Komentar penanda tahap nomor 11.

Baris 5

```
matriks_pixel = array_gambar[0:5, 0:5]
```

Mengambil baris indeks 0–4 dan kolom indeks 0–4 dari `array_gambar` (slicing, batas akhir tidak ikut). Hasilnya 25 pixel yang disimpan pada `matriks_pixel`. Cara ini sama dengan nomor 8, hanya areanya diperluas dari 3 × 3 menjadi 5 × 5.

Baris 7

```
print("Matriks pixel 5 × 5:")
```

Mencetak keterangan output.

Baris 8

```
print(matriks_pixel)
```

Mencetak matriks. Setiap elemen berupa satu pixel dengan tiga nilai RGB.

4. Hasil program

```
Matriks pixel 5 × 5:
[[[151 147 136]
  [145 141 130]
  [149 147 135]
  [154 152 140]
  [153 153 141]]

 [[152 148 137]
  [147 143 132]
  [147 145 133]
  [151 149 137]
  [150 150 138]]

 [[147 143 132]
  [145 141 130]
  [146 144 132]
  [150 148 136]
  [153 153 141]]

 [[149 145 134]
  [151 147 136]
  [150 148 136]
  [152 150 138]
  [153 153 141]]
...
  [146 144 132]
  [149 147 135]
  [152 150 138]
  [155 155 143]]]
```

Gambar 25. Hasil matriks pixel 5 × 5 pada nomor 11

5. Analisis hasil

Output menampilkan 5 baris dan 5 kolom pixel, dengan setiap pixel berisi tiga angka. Pixel pertama tetap `[151 147 136]` dan pixel di ujung kanan bawah bernilai `[155 155 143]`. Tiga baris dan tiga kolom pertamanya sama dengan hasil nomor 8, sehingga potongan 3 × 3 tersebut terbukti merupakan bagian dari matriks yang lebih besar ini. Dengan demikian, citra digital dapat dipandang sebagai matriks angka; seluruh foto tersimpan dalam `array_gambar`, sedangkan nomor 11 hanya memperlihatkan sebagian kecilnya agar mudah dibaca.

Codingan Nomor 12: Membentuk Kembali Gambar dari Angka

1. Tujuan codingan

Membentuk kembali gambar dari array angka lengkap (`array_gambar`) untuk membuktikan bahwa susunan angka pixel dapat menghasilkan kembali gambar yang dapat dilihat.

2. Screenshot kode program

```
# ========================================
# 12. MEMBENTUK KEMBALI GAMBAR DARI ANGKA
# ========================================

# Membentuk gambar dari array angka
gambar_rekonstruksi = Image.fromarray(array_gambar)

# Menampilkan gambar hasil rekonstruksi
plt.imshow(gambar_rekonstruksi)
plt.title("Gambar Hasil Rekonstruksi dari Array Angka")
plt.axis("off")
plt.show()
✓ 0.3s
```

Gambar 26. Kode program nomor 12 (rekonstruksi gambar)

3. Penjelasan setiap baris kode

Baris 1–3

```
# ====...
# 12. MEMBENTUK KEMBALI GAMBAR DARI ANGKA
# ====...
```

Komentar penanda tahap nomor 12.

Baris 5–6

```
# Membentuk gambar dari array angka
gambar_rekonstruksi = Image.fromarray(array_gambar)
```

`Image.fromarray()` adalah fungsi Pillow yang berkebalikan dengan `np.array()`: ia membentuk objek gambar dari sebuah array. Array yang dipakai adalah `array_gambar` secara lengkap (seluruh 2448 × 3264 pixel), bukan matriks contoh 5 × 5 dari nomor 11. Karena array berbentuk (tinggi, lebar, 3) dengan tipe `uint8`, Pillow mengenalinya sebagai gambar RGB. Hasilnya disimpan pada `gambar_rekonstruksi`.

Baris 8–9

```
# Menampilkan gambar hasil rekonstruksi
plt.imshow(gambar_rekonstruksi)
```

Menampilkan gambar hasil rekonstruksi pada Matplotlib.

Baris 10

```
plt.title("Gambar Hasil Rekonstruksi dari Array Angka")
```

Memberi judul pada gambar yang ditampilkan.

Baris 11

```
plt.axis("off")
```

Menyembunyikan sumbu koordinat agar tampilan rapi.

Baris 12

```
plt.show()
```

Menampilkan hasil visualisasi.

4. Hasil program



Gambar 27. Hasil gambar rekonstruksi pada nomor 12

5. Analisis hasil

Gambar yang dibentuk dari array angka tampil sama seperti foto pada nomor 3. Hal ini menunjukkan bahwa seluruh informasi visual foto memang tersimpan dalam susunan angka pixel: dari array tersebut gambar dapat dibentuk kembali. Pengamatan visual ini masih bersifat penglihatan, sehingga kesamaan nilainya diperiksa secara pasti pada nomor 13.

Codingan Nomor 13: Memeriksa Kesamaan Gambar Asli dan Hasil Rekonstruksi

1. Tujuan codingan

Membandingkan gambar asli dan gambar hasil rekonstruksi, baik secara visual maupun dengan membandingkan seluruh nilai pixelnya menggunakan `np.array_equal()`.

2. Screenshot kode program

```
# ========================================
# 13. MEMERIKSA KESAMAAN GAMBAR ASLI DAN HASIL REKONSTRUKSI
# ========================================

# Membandingkan array angka asli dengan hasil rekonstruksi
array_rekonstruksi = np.array(gambar_rekonstruksi)
gambar_sama = np.array_equal(array_gambar, array_rekonstruksi)

# Menampilkan kedua gambar berdampingan
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(gambar)
plt.title("Gambar Asli")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(gambar_rekonstruksi)
plt.title("Gambar Hasil Rekonstruksi")
plt.axis("off")

plt.tight_layout()
plt.show()

# Menampilkan hasil pemeriksaan kesamaan
print("Bentuk array gambar asli       :", array_gambar.shape)
print("Bentuk array hasil rekonstruksi :", array_rekonstruksi.shape)
print("Apakah kedua gambar identik?    :", gambar_sama)
0.7s
```

Gambar 28. Kode program nomor 13 (pemeriksaan kesamaan gambar)

3. Penjelasan setiap baris kode

Baris 1–3

```
# ====...
# 13. MEMERIKSA KESAMAAN GAMBAR ASLI DAN HASIL REKONSTRUKSI
# ====...
```

Komentar penanda tahap nomor 13.

Baris 5–6

```
# Membandingkan array angka asli dengan hasil rekonstruksi
array_rekonstruksi = np.array(gambar_rekonstruksi)
```

Mengubah gambar hasil rekonstruksi kembali menjadi array sehingga dapat dibandingkan dengan `array_gambar`.

Baris 7

```
gambar_sama = np.array_equal(array_gambar, array_rekonstruksi)
```

`np.array_equal()` memeriksa apakah dua array memiliki bentuk yang sama dan seluruh elemennya sama persis. Hasilnya `True` atau `False`, disimpan pada `gambar_sama`. Perbandingan ini mencakup semua nilai pixel, bukan hanya sampel.

Baris 9–10

```
# Menampilkan kedua gambar berdampingan
plt.figure(figsize=(12, 5))
```

Membuat kanvas gambar berukuran 12 × 5 inci agar dua gambar dapat ditampilkan berdampingan.

Baris 12–15

```
plt.subplot(1, 2, 1)
plt.imshow(gambar)
plt.title("Gambar Asli")
plt.axis("off")
```

`plt.subplot(1, 2, 1)` membagi kanvas menjadi 1 baris dan 2 kolom, lalu memilih panel pertama (kiri). Panel ini diisi gambar asli dengan judul "Gambar Asli" dan tanpa sumbu.

Baris 17–20

```
plt.subplot(1, 2, 2)
plt.imshow(gambar_rekonstruksi)
plt.title("Gambar Hasil Rekonstruksi")
plt.axis("off")
```

Memilih panel kedua (kanan) dan mengisinya dengan gambar hasil rekonstruksi berjudul "Gambar Hasil Rekonstruksi", juga tanpa sumbu.

Baris 22–23

```
plt.tight_layout()
plt.show()
```

`tight_layout()` mengatur jarak antarpanel agar judul dan gambar tidak saling menimpa, kemudian `show()` menampilkan kedua gambar.

Baris 25–28

```
# Menampilkan hasil pemeriksaan kesamaan
print("Bentuk array gambar asli        :", array_gambar.shape)
print("Bentuk array hasil rekonstruksi :", array_rekonstruksi.shape)
print("Apakah kedua gambar identik?    :", gambar_sama)
```

Mencetak bentuk array gambar asli, bentuk array hasil rekonstruksi, dan hasil pemeriksaan kesamaan (`gambar_sama`).

4. Hasil program



Gambar 29. Gambar asli dan gambar hasil rekonstruksi berdampingan (nomor 13)

```
Bentuk array gambar asli        : (2448, 3264, 3)
Bentuk array hasil rekonstruksi : (2448, 3264, 3)
Apakah kedua gambar identik?    : True
```

Gambar 30. Hasil pemeriksaan kesamaan pada nomor 13

5. Analisis hasil

Kedua gambar tampil sama dan kedua array memiliki bentuk yang sama, yaitu (2448, 3264, 3). Hasil `np.array_equal()` adalah `True`, artinya seluruh nilai pixel pada gambar asli dan gambar hasil rekonstruksi identik. Dengan demikian terbukti bahwa gambar yang dibentuk kembali dari array angka memiliki data pixel yang sama dengan gambar aslinya, tanpa ada nilai yang berubah.

Codingan Nomor 14: Kesimpulan Hasil Analisis Citra Digital

1. Tujuan codingan

Merangkum seluruh hasil analisis dalam satu keluaran ringkasan menggunakan nilai-nilai yang telah diperoleh pada tahap sebelumnya, mencakup ringkasan citra digital, ringkasan analisis koordinat pixel, dan ringkasan metadata GPS.

2. Screenshot kode program

```
# ========================================
# 14. KESIMPULAN HASIL ANALISIS CITRA DIGITAL
# ========================================

# Menghitung jumlah pixel dan kanal warna
jumlah_pixel = array_gambar.shape[0] * array_gambar.shape[1]
jumlah_kanal = array_gambar.shape[2]

# Menampilkan ringkasan informasi citra
print("RINGKASAN HASIL ANALISIS CITRA DIGITAL")
print("Ukuran gambar       :", gambar.size)
print("Dimensi array       :", array_gambar.shape)
print("Jumlah pixel        :", f"{jumlah_pixel:,}".replace(",", "."))
print("Jumlah kanal warna  :", jumlah_kanal)
print("Kanal warna         : Red (R), Green (G), Blue (B)")
print("Tipe data pixel     :", array_gambar.dtype)
print("Hasil rekonstruksi  :", gambar_sama)

# Menampilkan ringkasan analisis koordinat pixel
print("\nRINGKASAN ANALISIS KOORDINAT PIXEL")
print("Jumlah pixel dianalisis :", len(koordinat_pixel))

for baris, kolom in koordinat_pixel:

    # Mengambil nilai RGB dari setiap koordinat pixel
    r, g, b = array_gambar[baris, kolom]

    # Menampilkan koordinat dan nilai RGB
    print(f"Koordinat ({baris}, {kolom}) : RGB ({r}, {g}, {b})")
```

Gambar 31. Kode program nomor 14 bagian 1 (ringkasan citra dan koordinat pixel)

```
# Menampilkan ringkasan informasi GPS
print("\nRINGKASAN METADATA GPS")
print("Status metadata GPS :", "Tersedia" if gps else "Tidak tersedia")

if gps and 2 in gps and 4 in gps:

    # Menampilkan koordinat GPS dalam bentuk desimal
    print("Latitude (Lintang)  :", f"{lintang:.6f}°")
    print("Longitude (Bujur)   :", f"{bujur:.6f}°")
    print("Koordinat desimal   :", f"{lintang:.6f}, {bujur:.6f}")
    print("Google Maps         :", link_maps)

else:
    print("Koordinat GPS tidak lengkap atau tidak tersedia.")
✓ 0.0s
```

Gambar 32. Kode program nomor 14 bagian 2 (ringkasan metadata GPS)

3. Penjelasan setiap baris kode

Baris 1–3

```
# ====...
# 14. KESIMPULAN HASIL ANALISIS CITRA DIGITAL
# ====...
```

Komentar penanda tahap nomor 14.

Baris 5–7

```
# Menghitung jumlah pixel dan kanal warna
jumlah_pixel = array_gambar.shape[0] * array_gambar.shape[1]
jumlah_kanal = array_gambar.shape[2]
```

Menghitung kembali jumlah pixel dari tinggi × lebar array, sama dengan perhitungan pada nomor 7, dan mengambil jumlah kanal warna dari dimensi ketiga array, yaitu 3.

Baris 9–10

```
# Menampilkan ringkasan hasil analisis citra
print("RINGKASAN HASIL ANALISIS CITRA DIGITAL")
```

Mencetak judul ringkasan analisis citra.

Baris 11

```
print("Ukuran gambar       :", gambar.size)
```

Menampilkan ukuran gambar dalam urutan (lebar, tinggi): (3264, 2448).

Baris 12

```
print("Dimensi array       :", array_gambar.shape)
```

Menampilkan dimensi array dalam urutan (tinggi, lebar, kanal): (2448, 3264, 3).

Baris 13

```
print("Jumlah pixel        :", f"{jumlah_pixel:,}".replace(",", "."))
```

Menampilkan jumlah pixel dengan pemisah ribuan titik: 7.990.272.

Baris 14

```
print("Jumlah kanal warna  :", jumlah_kanal)
```

Menampilkan jumlah kanal warna: 3.

Baris 15

```
print("Kanal warna         : Red (R), Green (G), Blue (B)")
```

Mencetak teks tetap yang menyebut nama ketiga kanal. Baris ini tidak menghitung apa pun, hanya keterangan.

Baris 16

```
print("Tipe data pixel     :", array_gambar.dtype)
```

Menampilkan tipe data pixel: `uint8`.

Baris 17

```
print("Hasil rekonstruksi  :", gambar_sama)
```

Menampilkan hasil pemeriksaan kesamaan dari nomor 13 yang tersimpan pada `gambar_sama`: True.

Baris 19–21

```
# Menampilkan ringkasan analisis koordinat pixel
print("\nRINGKASAN ANALISIS KOORDINAT PIXEL")
print("Jumlah pixel dianalisis :", len(koordinat_pixel))
```

Mencetak judul ringkasan koordinat pixel; karakter `\n` menambah baris kosong agar terpisah dari ringkasan citra. `len(koordinat_pixel)` menghitung jumlah koordinat yang ditentukan pada nomor 9, yaitu 5.

Baris 23–29

```
for baris, kolom in koordinat_pixel:

    # Mengambil nilai RGB dari setiap koordinat pixel
    r, g, b = array_gambar[baris, kolom]

    # Menampilkan koordinat dan nilai RGB
    print(f"Koordinat ({baris}, {kolom}) : RGB ({r}, {g}, {b})")
```

Perulangan atas lima koordinat dari nomor 9. Pada setiap putaran, nilai R, G, dan B pixel diambil kembali dari `array_gambar`, kemudian koordinat dan nilai RGB-nya dicetak dalam satu baris menggunakan f-string.

Baris 31–33

```
# Menampilkan ringkasan informasi GPS
print("\nRINGKASAN METADATA GPS")
print("Status metadata GPS :", "Tersedia" if gps else "Tidak tersedia")
```

Mencetak judul ringkasan metadata GPS dan statusnya. Ekspresi `"Tersedia" if gps else "Tidak tersedia"` menghasilkan "Tersedia" jika variabel `gps` (dari nomor 6) berisi data, dan "Tidak tersedia" jika kosong.

Baris 35

```
if gps and 2 in gps and 4 in gps:
```

Memeriksa kembali kelengkapan koordinat dengan syarat yang sama seperti pada nomor 6, agar variabel `lintang`, `bujur`, dan `link_maps` hanya dipakai jika sudah dibuat.

Baris 37–41

```
    # Menampilkan koordinat GPS dalam bentuk desimal
    print("Latitude (Lintang)  :", f"{lintang:.6f}°")
    print("Longitude (Bujur)   :", f"{bujur:.6f}°")
    print("Koordinat desimal   :", f"{lintang:.6f}, {bujur:.6f}")
    print("Google Maps         :", link_maps)
```

Menampilkan lintang (-6.820098°), bujur (108.554794°), koordinat desimal gabungan (-6.820098, 108.554794), dan tautan Google Maps yang dibuat pada nomor 6. Tautan memakai nilai lintang dan bujur tanpa pembulatan.

Baris 43–44

```
else:
    print("Koordinat GPS tidak lengkap atau tidak tersedia.")
```

Pesan yang dicetak jika koordinat GPS tidak lengkap atau tidak tersedia. Pada foto ini cabang tersebut tidak dijalankan.

4. Hasil program

```
RINGKASAN HASIL ANALISIS CITRA DIGITAL
Ukuran gambar       : (3264, 2448)
Dimensi array       : (2448, 3264, 3)
Jumlah pixel        : 7.990.272
Jumlah kanal warna  : 3
Kanal warna         : Red (R), Green (G), Blue (B)
Tipe data pixel     : uint8
Hasil rekonstruksi  : True

RINGKASAN ANALISIS KOORDINAT PIXEL
Jumlah pixel dianalisis : 5
Koordinat (0, 0) : RGB (151, 147, 136)
Koordinat (0, 100) : RGB (180, 166, 157)
Koordinat (100, 100) : RGB (214, 206, 203)
Koordinat (200, 200) : RGB (142, 175, 154)
Koordinat (500, 500) : RGB (92, 99, 68)

RINGKASAN METADATA GPS
Status metadata GPS : Tersedia
Latitude (Lintang)  : -6.820098°
Longitude (Bujur)   : 108.554794°
Koordinat desimal   : -6.820098, 108.554794
Google Maps         : https://www.google.com/maps?q=-6.820097916666667,108.55479430555555
```

Gambar 33. Ringkasan hasil analisis, koordinat pixel, dan metadata GPS pada nomor 14

5. Analisis hasil

Ringkasan menegaskan bahwa foto berukuran 3264 × 2448 pixel disimpan sebagai array berdimensi (2448, 3264, 3) dengan 7.990.272 pixel, tiga kanal warna R, G, dan B, serta tipe data `uint8`. Nilai `True` pada hasil rekonstruksi menunjukkan bahwa array tersebut dapat membentuk kembali gambar yang identik dengan aslinya.

Ringkasan koordinat pixel menampilkan jumlah pixel yang dianalisis, yaitu 5, beserta nilai RGB-nya: (0, 0) bernilai RGB (151, 147, 136), (0, 100) bernilai (180, 166, 157), (100, 100) bernilai (214, 206, 203), (200, 200) bernilai (142, 175, 154), dan (500, 500) bernilai (92, 99, 68). Seluruh nilai ini sama dengan tabel pada nomor 9.

Pada bagian GPS, status metadata adalah Tersedia dengan lintang -6.820098°, bujur 108.554794°, dan koordinat desimal -6.820098, 108.554794. Tautan Google Maps menampilkan nilai tanpa pembulatan (-6.820097916666667, 108.55479430555555), sedangkan tabel pada nomor 6 membulatkannya menjadi 6 angka di belakang koma; keduanya berasal dari perhitungan yang sama. Seluruh hasil ini konsisten dengan nomor 4 sampai nomor 13 dan menjawab tujuan analisis.

C. Kesimpulan

Berdasarkan seluruh analisis, citra digital direpresentasikan sebagai array pixel dengan nilai RGB. Foto yang dianalisis berformat JPEG dengan ukuran 3264 × 2448 pixel, tersusun dari 7.990.272 pixel, dan memiliki tiga kanal warna, yaitu Red, Green, dan Blue. Setiap nilai kanal disimpan sebagai `uint8` dengan kedalaman 8 bit, sehingga satu pixel RGB menggunakan 24 bit.

Pengambilan sebagian pixel menunjukkan bahwa setiap pixel memiliki posisi dan tiga nilai angka, misalnya pixel (0, 0) bernilai R = 151, G = 147, dan B = 136. Nilai RGB dari lima koordinat pixel, yaitu (0, 0), (0, 100), (100, 100), (200, 200), dan (500, 500), berhasil dibaca dan menghasilkan nilai yang berbeda-beda. Nilai tersebut ditampilkan dalam tabel dan divisualisasikan sebagai blok warna, kemudian letak kelima pixel ditandai dengan titik merah dan label koordinat pada foto asli. Nilai pixel juga dapat ditampilkan sebagai matriks 5 × 5. Gambar kemudian dibentuk kembali dari array menggunakan `Image.fromarray()`. Pemeriksaan dengan `np.array_equal()` menghasilkan `True`, dan bentuk array kedua gambar sama, yaitu (2448, 3264, 3). Dengan demikian terbukti bahwa foto digital dapat direpresentasikan sebagai susunan pixel bernilai angka, dan susunan angka tersebut dapat dipakai untuk membentuk kembali gambar yang identik dengan gambar aslinya.

Selain data pixel, metadata EXIF foto memuat informasi perangkat (vivo V2252) dan waktu pengambilan, serta blok metadata GPS yang berhasil dibaca. Koordinat GPS dalam derajat, menit, dan detik (6° 49' 12,3525" S dan 108° 33' 17,2595" E) dikonversi menjadi derajat desimal, yaitu lintang -6.820098° dan bujur 108.554794°, dengan ketinggian 229 meter. Koordinat tersebut dipakai untuk membuat tautan Google Maps, sedangkan peta interaktif Folium tidak tampil karena library tersebut belum tersedia pada notebook. Dengan demikian, analisis ini menunjukkan bahwa Python dapat digunakan untuk memeriksa representasi numerik citra digital, merekonstruksi gambar dari array, dan membaca informasi tambahan berupa metadata GPS.